Coffee Order System — Full Roadmap

Node.js · Express · Supabase · Render · Plain HTML dashboard · No framework · No MVC · Just enough structure.

June 6 → June 29, 2025 · ~2–3 hrs/day · Your background: HTML/SCSS/JS, basic fetch, Node, Vercel · Shaky on async/await

What you're building — three connected systems

🤖 Messenger Bot

- → State machine order flow
- → Drink / milk / sweetness / building
- → Order # confirmation message
- → Checks daily limit before accepting
- → Auto-rejects when full or closed
- → "cancel" resets session

🖥️ Admin Dashboard (you only)

- → Today's order list — name, drink, building
- → Mark: pending → ready → done → cancelled
- → Set daily order limit
- → Master on/off switch
- → Edit pickup time + closed message
- → Daily revenue estimate

🗄️ Database (Supabase)

- → orders table — one row per order
- → config table — one row, your settings
- → Single source of truth
- → Real Postgres — actual SQL
- → Free tier, visual dashboard

One Express server does everything — handles Messenger webhooks AND serves your dashboard API AND serves your dashboard HTML as static files. No separate frontend server. No separate bot server. One repo, one deploy.

Project structure — "just enough", no MVC, no n-layer

No strict pattern. The rule: **things that do the same job live in the same file.** If you have to scroll past unrelated code to find something, split it. One function does one thing. If a function exceeds ~30 lines, it's doing two things.

```
coffee-bot/ ├── index.js ← Express setup only. Imports routes, starts server. 30 lines max. ├── .env ← ALL secrets. Never commit this. Ever. ├── .gitignore ← node_modules/ and .env — both lines, nothing else needed ├── package.json │ ├── routes/ │ ├── webhook.js ← GET /webhook (Meta verify) + POST /webhook (incoming msgs) │ └── api.js ← All /api/* routes. Admin only. Protected by secret header. │ ├── bot/ │ ├── handler.js ← State machine. The brain of the bot. handleMessage(id, text). │ ├── send.js ← sendMessage(recipientId, text). One job: call Messenger API. │ └── states.js ← const
```

```
STATES = { IDLE, ASK_NAME, ... }. Constants only. │ ├── db/ │ ├── client.js ← createClient() from supabase-js. Import this everywhere else. │ ├── orders.js ← saveOrder(), getTodayOrders(), countTodayOrders(), updateStatus() │ └── config.js ← getConfig(), updateConfig(). Reads/writes the single config row. │ └── dashboard/ ← Served as static files by Express. Plain HTML/CSS/JS — your territory. ├── index.html ← Order list table + settings panel. No framework. ├── style.css ← Your SCSS compiled, or plain CSS — whatever you prefer. └── app.js ← fetch() calls to /api routes. The one piece of frontend you write.
```

The `dashboard/` folder is purely frontend — HTML, CSS, JS you already know. Express serves it with one line: `app.use('/dashboard', express.static('dashboard'))`. No React, no Vite, no build step. You protect it with a password check in `app.js` that stores the admin secret in `sessionStorage`.

.env must never be committed. Write your `.gitignore` before your first `git init`. Leaked Supabase keys or Meta page tokens are a real problem — Meta can revoke your app, Supabase can lock the project.

Database — two tables, run once in Supabase SQL editor

orders — one row per order

COLUMN	TYPE	NOTES
`id` PK	`SERIAL`	auto-increment → becomes order number shown to customer
`sender_id`	`TEXT`	Messenger user ID — uniquely identifies who ordered
`name`	`TEXT`	what they typed as their name
`drink`	`TEXT`	'matcha_latte' | 'iced_latte' | 'dirty_matcha'
`milk`	`TEXT`	'oat' | 'cow'
`sweetness`	`TEXT`	'less' | 'regular' | 'extra'
`building`	`TEXT`	free text — wherever they said
`status`	`TEXT`	'pending' | 'ready' | 'done' | 'cancelled' — default 'pending'
`price`	`INTEGER`	in pesos — computed at order time based on drink + milk
`created_at`	`TIMESTAMPTZ`	DEFAULT NOW() — used to filter "today's" orders

config — always exactly one row — update, never insert again

COLUMN	TYPE	NOTES
`id`	`INTEGER`	always 1. One row. Always UPDATE, never INSERT after setup.
`daily_limit`	`INTEGER`	max orders accepted per day. You set this from dashboard.

		Default 15.
accepting_orders	BOOLEAN	master on/off. false = bot rejects all new orders immediately.
cutoff_message	TEXT	what bot replies when full or closed. You edit from dashboard.
pickup_time	TEXT	"8:00–8:15am, SJH corridor" — shown in every confirmation.

```
-- Paste and run in Supabase → SQL Editor

CREATE TABLE orders (
  id               SERIAL PRIMARY KEY,
  sender_id        TEXT NOT NULL,
  name             TEXT NOT NULL,
  drink            TEXT NOT NULL,
  milk             TEXT NOT NULL,
  sweetness        TEXT NOT NULL,
  building         TEXT NOT NULL,
  status           TEXT NOT NULL DEFAULT 'pending',
  price            INTEGER NOT NULL,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE config (
  id               INTEGER PRIMARY KEY DEFAULT 1,
  daily_limit      INTEGER NOT NULL DEFAULT 15,
  accepting_orders BOOLEAN NOT NULL DEFAULT true,
  cutoff_message   TEXT NOT NULL DEFAULT 'Sorry, orders are full for today. Try again tomo
rrow! 🙏',
  pickup_time      TEXT NOT NULL DEFAULT '8:00–8:15am, SJH corridor'
);

-- Seed the one config row
INSERT INTO config (id) VALUES (1);

-- Index so "today's orders" query is fast
CREATE INDEX idx_orders_date ON orders (DATE(created_at));
```

API routes — what your Express server exposes

METHOD	ROUTE	WHAT IT DOES	CALLER
		Meta verification handshake — echoes back the	

GET	`/webhook`	challenge token	Meta
POST	`/webhook`	Receives all incoming Messenger messages → calls handleMessage()	Meta
GET	`/health`	Returns 200 OK — keeps Render awake via UptimeRobot ping	UptimeRobot
GET	`/api/orders/today`	Returns array of today's orders sorted by id asc	Dashboard
PATCH	`/api/orders/:id/status`	Updates one order's status field. Body: `{ status: "ready" }`	Dashboard
GET	`/api/config`	Returns the single config row — limit, accepting, messages	Dashboard
PATCH	`/api/config`	Updates config. Body: any subset of config columns.	Dashboard

All `/api/*` routes need a simple auth check — one middleware function that reads `req.headers['x-admin-secret']` and compares it to your `ADMIN_SECRET` env var. Not JWT, not OAuth. A shared secret is fine at this scale. Your dashboard JS sends the header on every request.

Key code patterns — the shapes you'll write repeatedly

```
// ── bot/states.js ──────────────────────────────
const STATES = {
  IDLE:      'idle',
  ASK_NAME:  'ask_name',
  ASK_DRINK: 'ask_drink',
  ASK_MILK:  'ask_milk',
  ASK_SWEET: 'ask_sweet',
  ASK_BUILD: 'ask_build',
  DONE:      'done',
}
module.exports = STATES
```

```
// ── db/client.js ───────────────────────────────
const { createClient } = require('@supabase/supabase-js')
const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SERVICE_KEY  // service key, not anon key
)
module.exports = supabase
```

```
// ── db/orders.js ──────────────────────────────────
const supabase = require('./client')

async function saveOrder(orderData) {
  const { data, error } = await supabase
    .from('orders')
    .insert(orderData)
    .select()
    .single()
  if (error) throw error
  return data
}

async function countTodayOrders() {
  const today = new Date().toISOString().split('T')[0]
  const { count, error } = await supabase
    .from('orders')
    .select('*', { count: 'exact', head: true })
    .gte('created_at', `${today}T00:00:00`)
    .neq('status', 'cancelled')
  if (error) throw error
  return count
}

async function getTodayOrders() {
  const today = new Date().toISOString().split('T')[0]
  const { data, error } = await supabase
    .from('orders')
    .select('*')
    .gte('created_at', `${today}T00:00:00`)
    .order('id', { ascending: true })
  if (error) throw error
  return data
}

async function updateStatus(id, status) {
  const { error } = await supabase
    .from('orders')
    .update({ status })
    .eq('id', id)
  if (error) throw error
```

```
}

module.exports = { saveOrder, countTodayOrders, getTodayOrders, updateStatus }
```

```
// ── bot/handler.js — skeleton ──────────────────────
const STATES      = require('./states')
const { send }    = require('./send')
const { saveOrder, countTodayOrders } = require('../db/orders')
const { getConfig } = require('../db/config')

const sessions = {}  // in-memory. fine for MVP.

async function handleMessage(senderId, text) {
  const session = sessions[senderId] ?? { state: STATES.IDLE, data: {} }
  const t = text.trim()

  // "cancel" works from any state
  if (t.toLowerCase() === 'cancel') {
    sessions[senderId] = { state: STATES.IDLE, data: {} }
    await send(senderId, 'Order cancelled. Send anything to start again.')
    return
  }

  switch (session.state) {

    case STATES.IDLE: {
      const cfg   = await getConfig()
      const count = await countTodayOrders()
      if (!cfg.accepting_orders || count >= cfg.daily_limit) {
        await send(senderId, cfg.cutoff_message)
        break
      }
      sessions[senderId] = { state: STATES.ASK_NAME, data: {} }
      await send(senderId, "Hi! ☕ What's your name?")
      break
    }

    case STATES.ASK_NAME: {
      session.data.name  = t
      session.state      = STATES.ASK_DRINK
      sessions[senderId] = session
      await send(senderId,
```

```
        `Hey ${t}! What would you like?\n\n` +
        `1 — Iced matcha latte ₱89\n` +
        `2 — Iced latte ₱75\n` +
        `3 — Dirty matcha ₱105\n\n` +
        `Reply 1, 2, or 3.`)
      break
    }

    // ... ASK_DRINK, ASK_MILK, ASK_SWEET, ASK_BUILD follow same shape
    // each: validate input → save to session.data → advance state → send next question

    case STATES.DONE: {
      const cfg   = await getConfig()
      const order = await saveOrder({ ...session.data, sender_id: senderId })
      sessions[senderId] = { state: STATES.IDLE, data: {} }
      await send(senderId,
        `✅ Order #${order.id} confirmed!\n\n` +
        `📋 ${order.name}\n` +
        `☕ ${order.drink}\n` +
        `🥛 ${order.milk} · ${order.sweetness}\n` +
        `📍 ${order.building}\n` +
        `💳 ₱${order.price} — GCash on pickup\n\n` +
        `Pickup: ${cfg.pickup_time} 🙌`)
      break
    }
  }
}

module.exports = { handleMessage }
```

.env — every variable you need

```
# Messenger / Meta
PAGE_ACCESS_TOKEN=    # from Meta Developer → your app → Messenger → Access Tokens
VERIFY_TOKEN=         # you make this up — any random string — used once for webhook verif
y

# Supabase
SUPABASE_URL=         # Settings → API → Project URL
SUPABASE_SERVICE_KEY= # Settings → API → service_role key (not anon key)
```

```
# Admin dashboard protection
ADMIN_SECRET=         # you make this up — send as x-admin-secret header from dashboard JS

# Port (Render sets this automatically, but useful locally)
PORT=3000
```

service_role key bypasses Supabase row-level security. Never expose it to a browser or commit it to Git. Use it only in your server-side code. The anon key is for browsers. The service key is for servers.

23-day build plan — June 6 → June 29

2–3 hrs/day. The async primer first — everything else depends on it.

Days 1–2 · Jun 6–7 · ~2 hrs/day

Fix the weak spot first — async/await

- ☐ Read the async primer doc. Read it, don't skim.
- ☐ Exercise 1: write a `wait(ms)` function that wraps setTimeout in a Promise
- ☐ Exercise 2: fetch `jsonplaceholder.typicode.com/todos/1` in Node with async/await + try/catch
- ☐ Exercise 3: Express route that awaits `wait(1000)` then responds — see the delay
- ☐ Can you explain async/await out loud without reading notes? If yes, continue.

Days 3–4 · Jun 8–9 · ~2 hrs/day

Project init + Supabase + accounts

- ☐ Create Facebook Page → Meta Developer account → new app → Messenger product
- ☐ Create Supabase project, run both CREATE TABLE statements, seed config row
- ☐ `npm init -y` → install express dotenv axios @supabase/supabase-js
- ☐ Create folder structure exactly as shown above — empty files are fine
- ☐ Write `.gitignore` (node_modules + .env) → `git init` → first commit
- ☐ Write `.env` with placeholder values — confirm `process.env.PORT` loads

Days 5–6 · Jun 10–11 · ~2 hrs/day

Express server + webhook verification

- ☐ Build `index.js` : create Express app, mount routes, start server
- ☐ Build `routes/webhook.js` : GET /webhook verification handler
- ☐ Install ngrok → `ngrok http 3000` → paste URL into Meta dashboard
- ☐ Set your VERIFY_TOKEN in Meta dashboard + in .env → hit Verify
- ☐ Add GET /health → returns `{ status: 'ok' }`

- [ ] Goal: Meta dashboard shows webhook as "verified" ✓

Days 7–8 · Jun 12–13 · ~2 hrs/day

Receive messages + send replies

- [ ] Add POST /webhook in `routes/webhook.js` — log `req.body` raw, always return 200
- [ ] Send yourself a test message on Messenger → see it appear in terminal
- [ ] Build `bot/send.js` → `sendMessage(recipientId, text)` using axios
- [ ] Test: hardcode a reply in POST /webhook → confirm it appears in Messenger
- [ ] Goal: you message the bot, it echoes back anything ✓

Days 9–12 · Jun 14–17 · ~2–3 hrs/day

State machine — the whole bot brain

- [ ] Build `bot/states.js` constants
- [ ] Build `db/client.js` , `db/config.js` (getConfig), `db/orders.js` (all four functions)
- [ ] Build `bot/handler.js` — implement one state per session, test after each
- [ ] Wire limit + accepting_orders check in IDLE state
- [ ] Handle bad input on every state (type "matcha" instead of "1" → re-prompt)
- [ ] Wire `handler.js` into POST /webhook
- [ ] Goal: place a complete real order via Messenger → row appears in Supabase ✓

Days 13–15 · Jun 18–20 · ~2 hrs/day

Admin API routes

- [ ] Build `routes/api.js` with `adminAuth` middleware
- [ ] GET /api/orders/today → returns array
- [ ] PATCH /api/orders/:id/status → updates row
- [ ] GET /api/config + PATCH /api/config
- [ ] Test every route with Thunder Client (VS Code extension) — set x-admin-secret header
- [ ] Goal: you can read and update all data via API calls, no UI yet ✓

Days 16–19 · Jun 21–24 · ~2–3 hrs/day

Admin dashboard UI — your territory

- [ ] Wire Express to serve `dashboard/` as static: `app.use('/dashboard', express.static('dashboard'))`
- [ ] Build `dashboard/index.html` : order list table + settings panel layout
- [ ] Build `dashboard/app.js` : on load → fetch /api/orders/today → render rows
- [ ] Status buttons per row: click → PATCH /api/orders/:id/status → re-render

- ☐ Settings panel: daily limit input + toggle → PATCH /api/config on change
- ☐ Simple password gate: prompt for admin secret → store in sessionStorage → attach as header
- ☐ Auto-refresh orders every 30 seconds with setInterval
- ☐ Goal: open dashboard, see live orders, change a status, see it update in Supabase ✓

Days 20–22 · Jun 25–27 · ~2 hrs/day

Deploy to Render — go live

- ☐ Push to GitHub (triple-check .env is not included)
- ☐ Create Render Web Service → connect GitHub repo → set start command: `node index.js`
- ☐ Add all env vars in Render dashboard → deploy
- ☐ Update Meta webhook URL from ngrok → your Render URL
- ☐ Create UptimeRobot monitor → HTTP → your Render URL/health → every 5 mins
- ☐ Full end-to-end test on live URL — not localhost
- ☐ Goal: bot works, dashboard works, all on public URL ✓

Day 23 · Jun 28–29

Messenger Community + README + shipped

- ☐ Create Messenger Community manually — Announcements + Main chat channels
- ☐ Pin the bot's Messenger link in Announcements
- ☐ Invite 5–10 test users, run full order flow with real people
- ☐ Write GitHub README: what it is, screenshots, tech stack, how to run locally
- ☐ Screenshots of dashboard + Messenger flow for portfolio
- ☐ Shipped. ✓

Scope boundary — protect the deadline

✓ Shipping June 29

- ✓ Full Messenger order flow
- ✓ Daily limit from config
- ✓ Master on/off switch
- ✓ Order list dashboard
- ✓ Status updates per order
- ✓ Settings panel
- ✓ Deployed + live on Render
- ✓ Messenger Community set up

✕ v2 — after it's live

- ✕ GCash / payment integration
- ✕ Order history across days
- ✕ Revenue analytics / charts
- ✕ Customer order editing
- ✕ Auto daily announcements
- ✕ Dirty tablea oat milk menu item
- ✕ Mobile-optimized dashboard
- ✕ Any kind of auth beyond header secret

Resources — only what you need, in order

RESOURCE	WHAT TO READ	WHEN
javascript.info	Promises, async/await chapters only	Days 1–2
Meta Messenger Platform docs	"Get Started" + "Webhook" sections only. Ignore everything else for now.	Days 5–6
Express.js docs	Routing + serving static files. Two pages total.	Days 5–6
Supabase JS docs	`.insert()`, `.select()`, `.update()`, `.eq()`, `.gte()` only.	Days 9–12
ngrok docs	Just run: `ngrok http 3000`. That's it.	Days 5–6
Thunder Client (VS Code)	Install from Extensions. Use it instead of Postman — stays inside VS Code.	Days 13–15
Render docs	"Deploy a Node.js app" — one page, 5 mins.	Days 20–22

This is a complete full-stack app: third-party API integration, stateful backend logic, real database, protected REST API, and a functional frontend you built. At 2nd year CS level this is a legitimate portfolio piece — especially because it had real users and solved a real problem you had. Ship the MVP. Add features after it's live.